

UDESC

Departamento de Ciência da Computação

Estrutura de Dados 2

**Análise comparativa da complexidade algorítmica de
árvores AVL, rubro-negras e B**

Alunos: Gabriela Alves Ilezyszyn, Heitor Linhares Caetano e Pedro Henrique Dias
Professor: Allan Rodrigo Leite

Junho
2026

Conteúdo

1	Objetivo do Trabalho	1
2	Metodologia	2
2.1	Contagem de comparações	2
3	Implementação	3
3.1	Árvore AVL	3
3.2	Árvore Rubro-negra	3
3.3	Árvores B	4
4	Resultados	6
4.1	Inserção de Chaves	6
4.2	Remoção de Chaves	7

1 Objetivo do Trabalho

O objetivo do trabalho é analisar e comparar a complexidade computacional das operações de inserção e remoção de nós em árvores AVL, rubro-negras e B, considerando também o custo de balanceamento. Para isso, foram usados conjuntos de dados aleatórios com tamanhos variando de 1 a 10 mil. Os resultados foram apresentados em gráficos que relacionam o esforço temporal das operações, seguidos de uma discussão sobre o comportamento e a eficiência das diferentes estruturas apresentadas.

2 Metodologia

Para a criação dos conjuntos aleatórios, foi criado um vetor de tamanho 10 mil em que cada posição armazena o próprio índice. Após isso, é usado um [algoritmo de embaralhamento](#) chamado Fisher-Yates que randomiza o vetor com base em valores aleatórios obtidos com a função *rand*.

A partir desse vetor, o experimento foi construído de forma a avaliar o crescimento das estruturas de maneira incremental. Em cada iteração, as árvores são criadas do zero. Em seguida, os elementos são inseridos em passos fixos (400 elementos, no nosso caso) até atingir o tamanho máximo (10 mil).

Em cada ponto de amostragem, o custo computacional das operações é acumulado pelas inserções realizadas naquela repetição. Dessa forma, cada ponto do gráfico representa o comportamento de crescimento da estrutura ao longo de sua construção incremental.

Para a remoção, no mesmo ponto, as chaves presentes nas estruturas são copiados para um vetor auxiliar e embaralhados novamente, fazendo com que haja uma ordem aleatória na remoção. Em seguida, todas as chaves são removidas das árvores previamente criadas, com o custo sendo contabilizado. Assim, é possível medir o comportamento de inserção e remoção aproveitando uma mesma árvore, ao mesmo tempo que mantemos diferentes tamanhos de entrada.

Como os resultados são divididos pelo número de repetições realizadas + o tamanho de suas respectivas árvores, os gráficos finais mostram a média de 30 comparações realizadas em cada tamanho.

2.1 Contagem de comparações

Para estabelecer um método de comparação entre a complexidade das diferentes árvores, se faz necessário o uso da comparação como unidade de esforço computacional. Comparações são todas as partes do código-fonte que estabelecem uma mudança de fluxo condicional ao sistema. Na linguagem C, elas são invocadas em toda aparição das *keywords* `if`, `for`, `while`, `do{...}while`, além do operador ternário `?`.

Contudo, nem toda ocorrência desses termos é contabilizada para os fins deste trabalho. A justificativa para isso é de que nem toda comparação no código é inerente à estrutura analisada, sendo muitas vezes apenas parte da organização do fluxo do programa. Portanto, seguindo um modelo parecido com o que foi apresentado em sala durante a verificação de complexidade de diferentes buscas em vetores, contabilizamos como comparações apenas aquelas que acontecem entre chaves ou entre chaves e valores internos da

estrutura.

Assim, não são contabilizadas como parte da métrica operações de verificação de ponteiros, como `if(raiz == NULL)` ou `\verbif(!raiz)|`, usadas para controle de fluxo ou verificações de alocação de memória.

Essas operações são produtos apenas de decisões de implementação e gerenciamento de fluxo em linguagem C, e não representam o custo "matemático" do algoritmo. Portanto, foram contabilizados apenas os testes de relação entre chaves e de elementos armazenados, esse sim próprios a cada estrutura de dados.

Dessa forma, a métrica utilizada foca exclusivamente no custo de comparação entre elementos armazenados.

3 Implementação

3.1 Árvore AVL

A árvore AVL foi implementada de maneira similar a demonstrada em aula, porém com uma pequena diferença: cada nó também armazena um inteiro que representa sua altura. Essa altura é atualizada a cada inserção ou remoção e é usada no cálculo do fator de balanceamento, que se torna muito mais simples com essa informação já em mãos, sendo apenas a diferença entre as alturas das subárvores esquerda e direita ($FB = altura(esquerda) - altura(direita)$). Caso o valor absoluto do fator de balanceamento atinja 2, as rotações simples ou duplas são acionadas para reestruturar a árvore.

Esse armazenamento das alturas de cada nó ajuda significativamente na eficiência da árvore, sendo o padrão na maioria das implementações disponíveis na *internet*. Se a estrutura precisasse percorrer recursivamente por todas suas subárvores para descobrir suas alturas a cada inserção ou remoção, a operação de balanceamento teria maior complexidade, sendo $O(n)$ no pior dos casos, diminuindo a vantagem de se usar uma árvore auto-balanceada.

Quando cada nó armazena explicitamente sua altura, o cálculo do fator de balanceamento se torna uma operação de esforço constante ($O(1)$).

3.2 Árvore Rubro-negra

A árvore Rubro-Negra foi implementada garantindo as propriedades fundamentais de balanceamento através da atribuição de cores (vermelho ou preto) a cada nó. Diferente da AVL, a Rubro-Negra assegura que o caminho

mais longo da raiz até uma folha não seja maior que o dobro do caminho mais curto, mantendo a altura da árvore em $O(\log n)$.

A estrutura mantém a consistência da árvore através de regras rigorosas: Todo nó é ou vermelho ou preto, a raiz é sempre preta, e não podem existir dois nós vermelhos adjacentes. Sempre que uma inserção ou remoção viola essas propriedades, a árvore utiliza um conjunto de rotações e, crucialmente, a alteração das cores dos nós para restaurar o equilíbrio.

Esse armazenamento da cor em cada nó é o detalhe que confere a eficiência desta estrutura. Enquanto a AVL prioriza uma busca mais rápida, a Rubro-Negra é otimizada para cenários de alta rotatividade de dados. Como o rebalanceamento é feito através da mudança das cores e rotações são menos frequentes que na AVL, a árvore tende a apresentar um desempenho superior em sistemas onde inserções e remoções são tão frequentes quanto buscas.

3.3 Árvores B

A implementação da Árvore B foi projetada para ser flexível em relação ao seu parâmetro de ordem, que define a capacidade de ocupação de cada nó. Diferente das árvores AVL e Rubro-Negra, que são estruturalmente binárias, a Árvore B agrupa múltiplas chaves e ponteiros em um único bloco de memória. Na implementação, a capacidade máxima de um nó foi definida como $2 \times \text{ordem chaves}$.

Para otimizar o desempenho computacional, a busca pela posição correta de uma chave dentro do vetor de um nó não é feita de forma sequencial, mas sim através de uma *busca binária* (implementada na função `pesquisa_binaria`). Embora árvores B de maior ordem (como a ordem 10) possuam menos níveis (reduzindo a descida na árvore), elas acomodam vetores maiores em cada nó, o que eleva a quantidade de comparações feitas pela busca binária local.

A operação de adição ocorre com a localização da folha adequada e a inserção ordenada da nova chave. Caso a quantidade de chaves do nó ultrapasse o limite máximo estabelecido, é disparada a condição de transbordo (*overflow*). Nesse cenário, o algoritmo realiza a operação de *split* (divisão): o nó é particionado em dois e a chave mediana é promovida para o nó pai. Esse processo pode ocorrer em cascata, subindo pela estrutura e, no limite, dividindo a raiz e aumentando a altura da árvore.

Por fim, a operação de remoção foi realizada de forma que, para evitar lacunas no meio da árvore, caso a exclusão seja solicitada em um nó interno, a chave é previamente substituída por seu sucessor imediato (que sempre reside em um nó folha). A remoção física ocorre exclusivamente nas folhas. Após

a exclusão, caso o número de chaves do nó caia abaixo do mínimo aceitável (condição de *underflow*, que equivale à ordem da árvore), o algoritmo tenta realizar o empréstimo de uma chave de um de seus nós irmãos adjacentes através do pai. Se ambos os irmãos estiverem no limite mínimo, é realizada a operação de fusão (*merge*) do nó com seu irmão e a chave separadora do pai, um processo que diminui o tamanho do nó pai e pode propagar o *underflow* até a raiz da árvore.

4 Resultados

A partir dos arquivos `.csv` criados na função `main()`, foram criados dois gráficos, preparados através de um script do *software* `gnuplot`. Vamos analisar cada um deles:

4.1 Inserção de Chaves

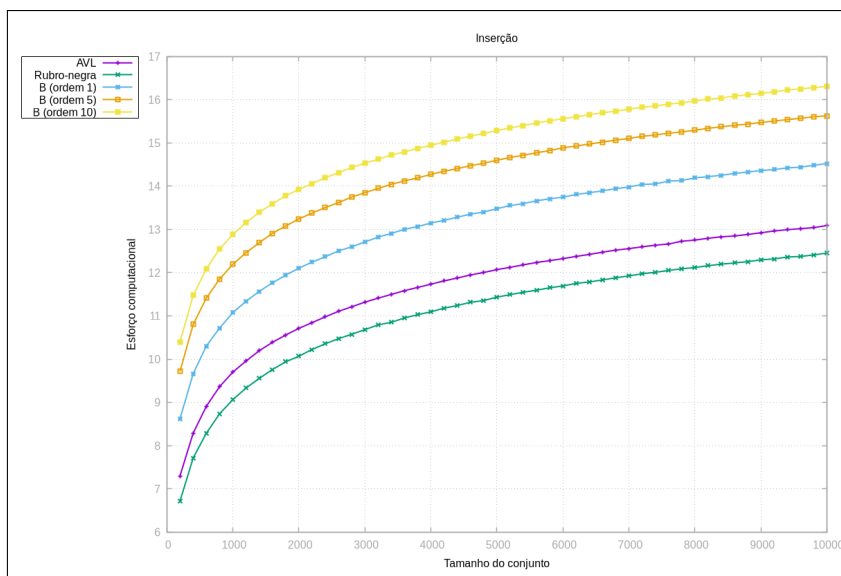


Figura 1: Gráfico

Aqui podemos observar que as árvores B geram maior esforço computacional durante a inserção, de modo que quanto maior a ordem da árvore, maior o esforço. Na teoria, as árvores B são mais eficientes para reduzir acessos à memória secundária (HD, SSD etc.), pois acabam sendo mais largas e rasas. Porém, quando é medido apenas esforço computacional, nesse caso comparações entre chaves realizadas na memória principal (RAM), elas acabam sendo penalizadas por alguns fatores.

O primeiro é o da busca binária que ocorre em cada nó. Para descer um único nível em direção às suas folhas, as AVL e Rubro-negras fazem apenas uma comparação de chaves. Já as B, como armazenam múltiplos elementos em um nó, realizam várias operações em um loop `while` interno.

Ademais, quando um nó transborda, a chave promovida precisa ser reinserida no pai. Na nossa implementação, isso chama uma nova busca binária no pai, o que ajuda a acumular ainda mais comparações.

A ordem da árvore também importa nesse caso. Quanto maior ela for, maior é a densidade das chaves armazenadas no vetor interno de cada nó. Assim, a busca binária fará mais iterações.

Também é notável que a AVL exige mais esforço que a Rubro-Negra. Isso ocorre pois a AVL é mais rígida em seu critério de balanceamento, fazendo com que ela precise recalcular seus dados internos mais frequentemente que a Rubro-negra, que acabou "ganhando" a disputa de inserção.

4.2 Remoção de Chaves

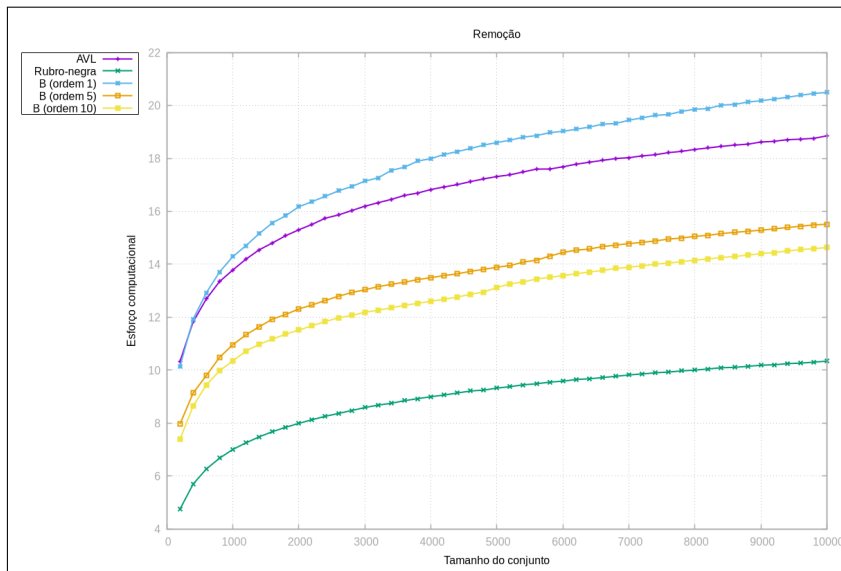


Figura 2: Gráfico

Aqui notamos algumas coisas interessantes. A árvore B "disparou" como a pior de todas. Isso ocorre porque ela armazena no máximo 2 chaves por nó, e por ter nós pequenos, ela acaba tendo uma alta taxa de underflow, além de ter o overhead da função `trata_subfluxo`.

Também notamos que a AVL fica em segundo lugar. Isso vem de um motivo similar ao mencionado na inserção; a árvore AVL é mais estrita, e acaba realizando mais comparações em suas operações. Além disso, podemos ver que as árvores B5 e B10 melhoraram, pois, nesse caso, seus nós com muitas chaves são uma vantagem, já que a remoção de um elemento raramente causa um subfluxo. Outro fator é de que o algoritmo quase não precisa recorrer a fusões de chaves entre irmãos, "desviando" do custo da função `trata_subfluxo`.

A Observação mais diferente e inesperada é a da árvore rubro-negra ser considerada a mais eficiente para remoção. Compreendemos isso como uma consequência de sua implementação, onde a função `remove` faz apenas um `while` simples até encontrar o nó desejado, e de que o balanceamento é realizado basicamente apenas pelas cores dos nós, que não foram contabilizados. Como a rubro-negra não faz comparações entre valores numéricos, ela acabou sendo a mais eficiente para fins do nosso teste.